UNIDAD 6.2

Diseño de Sistemas de Información

Tipos de pruebas: de la unidad a la aceptación



REPASO

Pruebas: repasemos

1 Un caso exitoso encuentra un error

Es una inversión: cada error hallado ahora es un costo evitado después.

2 Probar es destructivo

Se busca provocar fallas, no confirmar que todo anda bien.

3 No se puede probar todo

Psicología (¿quién prueba?) y economía (¿cuántos casos y para qué?).

4 Caja negra y caja blanca

Casos derivados de la especificación o de la estructura interna del código.

MAPA GENERAL

Niveles, tipos y técnicas

NIVELES

¿Sobre qué alcance se prueba?

- Unidad
- Integración
- Sistema
- Aceptación

TIPOS

¿Qué atributo se verifica?

- Funcional
- Humo y regresión
- Performance y esfuerzo
- Seguridad y usabilidad

TÉCNICAS

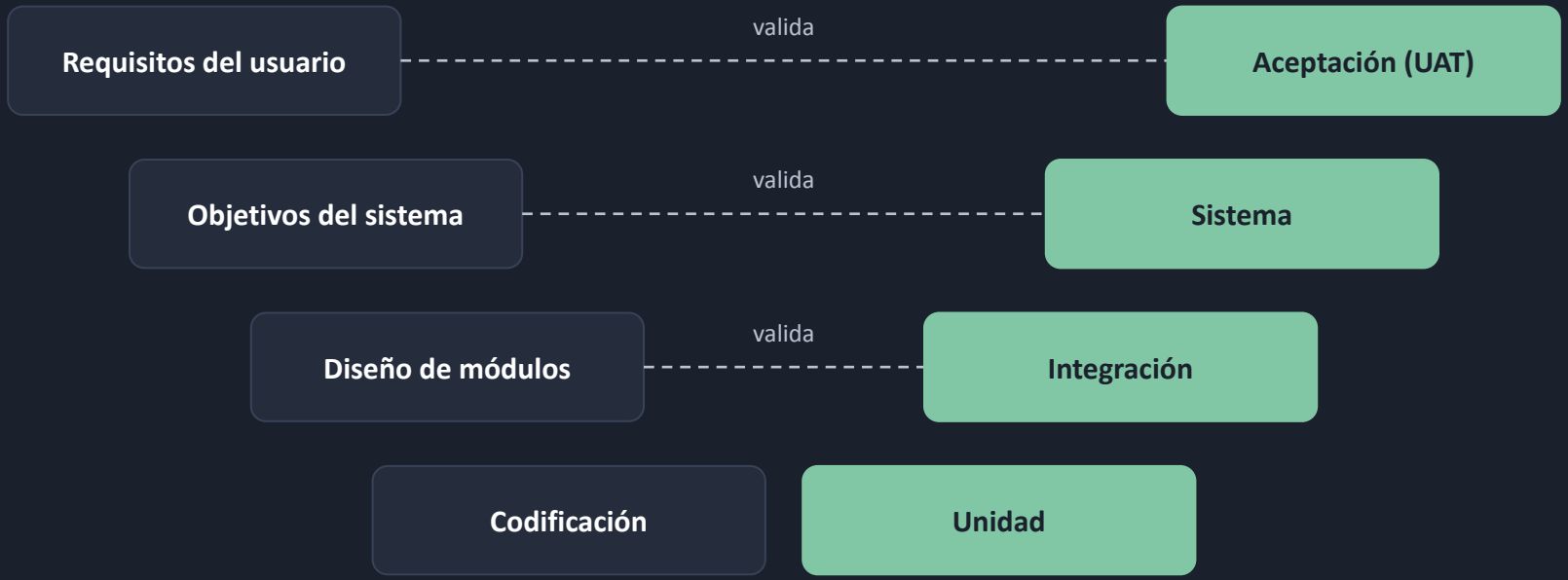
¿Cómo se diseñan los casos?

- Partición de equivalencia
- Valores límite
- Conjetura de errores
- Cobertura lógica

Un mismo caso se describe con las tres miradas: una prueba de integración (nivel), de regresión (tipo), diseñada con valores límite (técnica).

MAPA GENERAL

El modelo en V



Cada nivel de prueba se contrasta con la etapa que le corresponde.

DESARROLLO

PRUEBAS

NIVEL · UNIDAD

Pruebas de unidad (unit testing)

QUÉ ES

Probar clases, procedimientos, funciones o subrutinas de forma particular, aislada del resto.

Foco en una porción pequeña

La atención está puesta en poco código.

Facilita el debugging

Si falla, el sospechoso está acotado.

Permite paralelizar

Varias personas prueban distintas unidades a la vez.

Las pruebas de unidad se piensan como caja blanca, aunque sus datos de entrada pueden salir de técnicas de caja negra.

NIVEL · UNIDAD

Una buena prueba de unidad

1

Automatizables

Se ejecutan solas en cada cambio, dentro del pipeline de CI/CD.

2

Independientes

No dependen de bases de datos ni de sistemas externos: se usan mocks o stubs.

3

Documentan

Describen el comportamiento esperado de la unidad con ejemplos ejecutables.

REGLA MNEMOTÉCNICA: FIRST

F

Fast
rápidas

I

Independent
independientes

R

Repeatable
repetibles

S

Self-validating
se autoevalúan

T

Timely
a tiempo

Ejemplo: una prueba con [Fact]

```
using Xunit;

public class Calculadora
{
    public int Sumar(int a, int b) => a + b;
}
```

```
public class CalculadoraTests
{
    [Fact]
    public void Sumar_DosNumeros_RetornaSumaCorrecta()
    {
        // Arrange
        var calc = new Calculadora();
        // Act
        var resultado = calc.Sumar(2, 3);
        // Assert
        Assert.Equal(5, resultado);
    }
}
```

1 **Arrange**

Preparar el objeto y los datos.

2 **Act**

Ejecutar la unidad bajo prueba.

3 **Assert**

Verificar el resultado esperado.

Nombre: Método_Escenario_Resultado. El nombre ya describe el caso.

Ejemplo: una prueba, varios casos

```
public class StringHelper
{
    public bool EsPalindromo(string texto)
    {
        var invertido = new string(
            texto.Reverse().ToArray());
        return texto.Equals(invertido,
            StringComparison.OrdinalIgnoreCase);
    }
}
```

```
[Theory]
[InlineData("oso", true)]
[InlineData("Hola", false)]
public void EsPalindromo_PruebasVariosCasos(
    string palabra, bool esperado)
{
    var helper = new StringHelper();                // Arrange
    var resultado = helper.EsPalindromo(palabra);   // Act
    Assert.Equal(esperado, resultado);              // Assert
}
```

¿Qué casos faltan?

`""`

texto vacío

`"Oso"`

mayúsculas

`"a"`

un solo carácter

`"anita lava la tina"`

con espacios

`null`

entrada nula

Aislar la unidad: mocks y stubs




```
var repo = new Mock<IClienteRepository>();
repo.Setup(r => r.Obtener(1))
    .Returns(new Cliente { Id = 1, Nombre = "Ana" });

var servicio = new ClienteService(repo.Object);
var nombre = servicio.NombreCompleto(1);

Assert.Equal("Ana", nombre);
repo.Verify(r => r.Obtener(1), Times.Once);
```

Stub

Devuelve datos preparados.

Mock

Además verifica cómo fue usado.

NIVEL · UNIDAD

La pirámide de pruebas



Hacia arriba

Más costo, más tiempo de ejecución, más fragilidad.

Hacia abajo

Más velocidad y mejor ubicación de la falla.

Una pirámide invertida (casi todo a mano o por la interfaz) hace lento y caro cada cambio.

NIVEL · INTEGRACIÓN

Pruebas de integración

Verifican que módulos o componentes ya probados de forma aislada funcionan correctamente juntos.

Contratos de datos

Formato, tipos y nombres de campos que no coinciden.

Orden y estado

Llamadas en secuencia incorrecta o con estado inesperado.

Manejo de errores

Timeouts, códigos de error y reintentos mal tratados.

Configuración

Permisos, versiones, zonas horarias, codificación.

Ejemplo: frontend Angular, API en C# y SharePoint vía Microsoft Graph: cada flecha entre ellos es un punto de integración.

Incremental versus no incremental

NO INCREMENTAL (BIG-BANG)

¿Probamos cada módulo de forma independiente y luego los combinamos para formar el programa?

- Permite probar los módulos en paralelo
- Si falla al combinar, ¿qué módulo es el culpable?

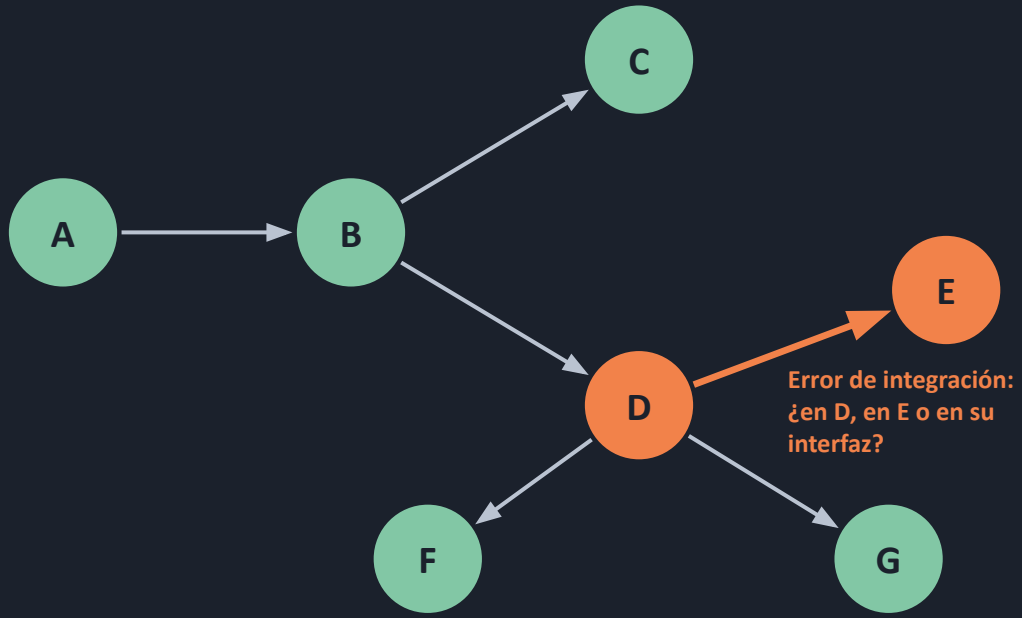
INCREMENTAL

¿Combinamos el siguiente módulo con el conjunto ya probado, y recién entonces lo probamos?

- La falla apunta al último módulo agregado
- Los errores de interfaz aparecen antes
- Los primeros módulos se prueban más veces

En general se prefiere la prueba incremental.

Ejemplo no incremental



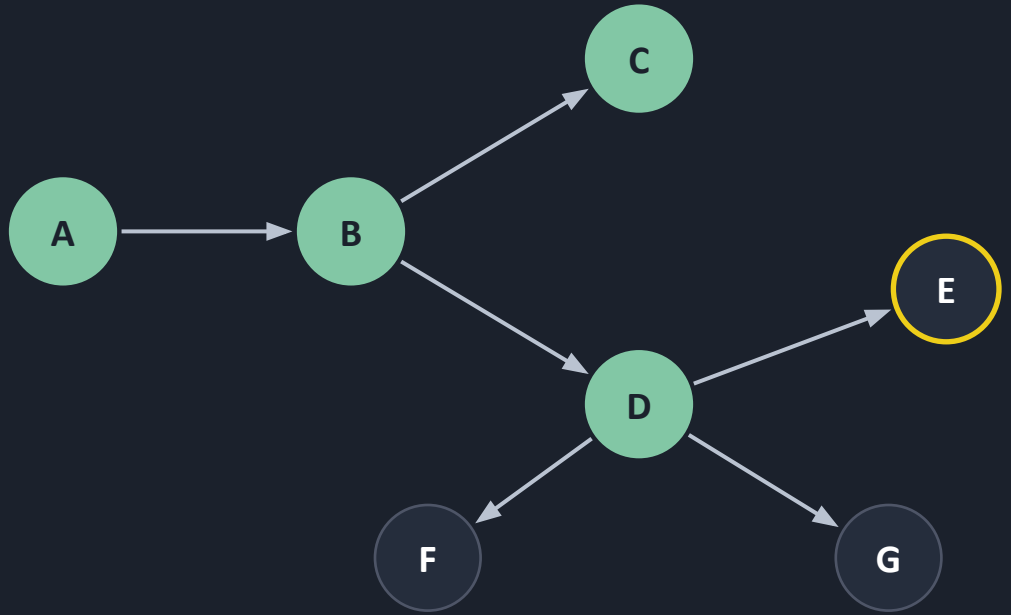
Paso 1

Cada módulo se prueba solo, con sus drivers y stubs.

Paso 2

Se combinan todos y se prueba el programa completo.

Ejemplo incremental



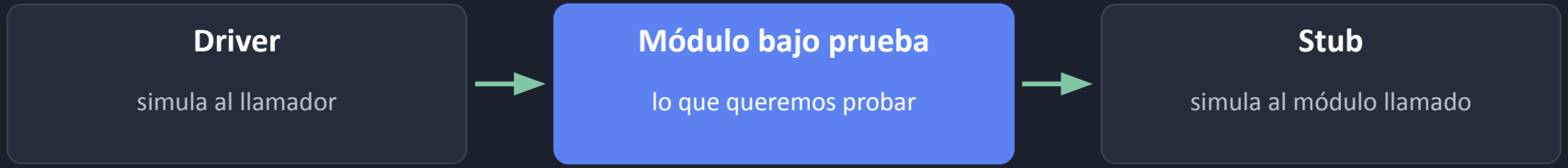
Paso 4: ya integrados A, B, C, D. Sigue E.

SECUENCIA DE PRUEBAS

- A
- AB
- ABC
- ABCD
- ABCDE
- ABCDEF
- ABCDEFG

Si falla un paso, el sospechoso es el último módulo agregado.

Descendente y ascendente



Descendente (top-down)

Se empieza por A y se bajan los niveles, usando stubs para los módulos aún no integrados.

- Prueba temprano la lógica de control
- Los stubs pueden ser complejos

Ascendente (bottom-up)

Se empieza por las hojas (C, E, F, G) y se sube, usando drivers que las invocan.

- No requiere stubs; módulos base probados a fondo
- El programa completo recién existe al final

TIPO · FUNCIONAL

Pruebas de función

Verifican que el software cumpla con las funcionalidades especificadas en los requisitos.

1

Entradas válidas

Cada función responde correctamente.

2

Entradas inválidas

Cada función las maneja de forma adecuada.

3

Requisitos del sistema

Cada función cumple con lo especificado.

Generalmente se hacen con técnicas de caja negra, porque se enfocan en entradas y salidas.

TIPO · FUNCIONAL

Ejemplo: carrito de compras

Precondición	Acción	Resultado esperado
Carrito vacío	Agregar producto de $1.000	Aparece en el carrito; total $1.000
Carrito con 1 producto	Agregar el mismo producto	Cantidad 2; total $2.000
Carrito con 1 producto	Quitar el producto	Desaparece; total $0
Producto con stock 5	Agregar 6 unidades	Rechaza: stock insuficiente
Producto con stock 5	Agregar 5 unidades	Acepta; total = 5 × precio

Agregar y eliminar un producto son las dos pruebas mínimas del ejemplo original; el resto sale de clases inválidas y valores límite.

TIPO · FUNCIONAL

¿Cómo documentamos las pruebas?

1. ¿Tiene un nombre definido?
2. ¿Está vinculado a los casos de uso?
3. ¿Tiene historial de cambios?
4. ¿Está claro el objetivo?
5. ¿Tiene las precondiciones necesarias?
6. ¿Cada paso tiene resultado esperado?
7. ¿Se documenta el resultado de la ejecución?
8. ¿Se registra quién lo ejecutó?
9. ¿Se registran los ciclos ejecutados?

Resaltado: sin resultado esperado por paso no hay caso de prueba, solo una secuencia de acciones.

TIPO · FUNCIONAL

Pruebas de humo (smoke testing)

QUÉ SON

Un conjunto mínimo de pruebas que verifica rápido si la aplicación funciona en lo básico y es estable para seguir probándola.

Detectan fallos graves muy temprano en el ciclo de pruebas.

Evitan perder tiempo probando en detalle un sistema que no arranca.

Rápidas y frecuentes se ejecutan en horas, son superficiales.

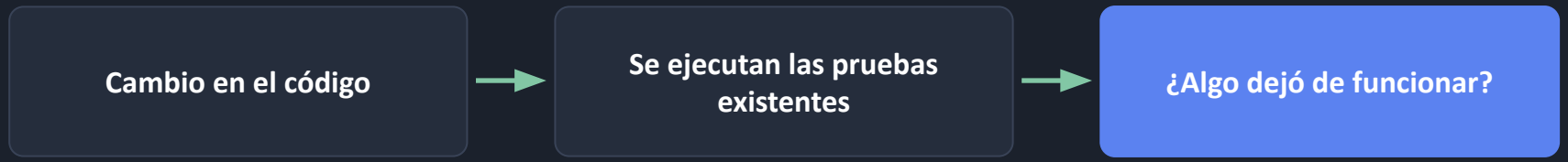
EJEMPLO: tienda online

1. Abre la aplicación
2. Inicia sesión
3. Busca un producto
4. Agrega al carrito

TIPO · FUNCIONAL

Pruebas de regresión (regression testing)

Verifican que los cambios (correcciones, mejoras, funcionalidades nuevas) no rompen lo que antes funcionaba.



Objetivo

Mantener la estabilidad del sistema.

Cuándo

Después de cada corrección, mejora o integración.

Cómo

Automatizadas y priorizadas por riesgo.

Ejemplo: se corrige el cálculo del descuento, y se vuelven a correr las pruebas del carrito y de la facturación.

TIPO · FUNCIONAL

Humo, sanidad y regresión

Tipo	Pregunta	Alcance	Cuándo
Humo	¿Arranca y funciona lo básico?	Amplio y superficial	En cada versión o despliegue
Sanidad	¿El cambio puntual anda?	Acotado a un área	Después de un arreglo
Regresión	¿Se rompió algo que andaba?	Amplio y profundo	Después de cada cambio

Las tres son candidatas a automatizarse en el pipeline de integración continua.

NIVEL · SISTEMA

Pruebas de sistema

OBJETIVO

Comparar el sistema con sus objetivos originales.

Los casos de prueba se diseñan a partir del análisis de los objetivos.

La más difícil

No hay una técnica única: depende de los objetivos.

La peor interpretada

No es repetir las pruebas funcionales o de integración.

Entorno parecido a producción

Se prueba el sistema completo, no sus partes.

Incluye requisitos funcionales y no funcionales (performance, seguridad, usabilidad).

NIVEL · SISTEMA

Tipos de pruebas de sistema

1

Alcance

¿Está todo lo prometido?

2

Volumen

Muchos datos

3

Esfuerzo

Picos de carga

4

Facilidad de uso

Amigabilidad

5

Seguridad

Vulnerabilidades

6

Performance

Objetivos de tiempo

7

Otros tipos

Configuración, recuperación…

Pruebas de alcance

Verificar que cada funcionalidad mencionada en los objetivos fue realmente implementada.

Objetivo del sistema	¿Implementado?	Evidencia
“El usuario debería especificar un rango de direcciones…”	Sí	Pantalla de búsqueda, campos desde y hasta
Exportar el listado a PDF	**No**	Falta: no existe el botón ni el servicio
Registrar quién modificó cada reserva	**Parcial**	Se guarda la fecha pero no el usuario

Este tipo de prueba puede efectuarse sin computadora: es una revisión contra los objetivos.

SISTEMA · VOLUMEN

Pruebas de volumen

Someter al sistema a un gran volumen de datos para demostrar que no puede manejar los volúmenes especificados en sus objetivos.

Un compilador recibe un programa enorme.

Una base de datos recibe millones de registros.

Un reporte se genera sobre diez años de historia.

Un archivo de carga masiva de varios gigabytes.

QUÉ OBSERVAR

- Tiempos de respuesta de consultas
- Límites de almacenamiento
- Paginación y desbordes
- Errores por truncamiento

Pruebas de performance

Verifican si el sistema cumple los objetivos de performance establecidos.

Ejemplo: con una carga de x usuarios, el tiempo de respuesta debe ser de x segundos.

Tiempo de respuesta

Cuánto tarda cada operación; se mira el promedio y los percentiles.

Rendimiento (throughput)

Cuántas operaciones por segundo procesa.

Uso de recursos

CPU, memoria, disco y red bajo carga.

Estabilidad

Si se degrada o se cae con carga sostenida.

Volumen, esfuerzo y performance

Tipo	¿Qué se aumenta?	Pregunta	Ejemplo
Volumen	La cantidad de datos	¿Aguanta el volumen especificado?	10 millones de registros
Esfuerzo	La carga en poco tiempo	¿Qué pasa en un pico o más allá del límite?	5.000 usuarios en 1 minuto
Performance	La carga prevista	¿Cumple los tiempos objetivo?	500 usuarios, respuesta en menos de 2 s

Pruebas de esfuerzo: simulan un pico de volumen en un período corto. Con JMeter se simulan múltiples usuarios sobre una aplicación web.

Pruebas de facilidad de uso

Se relaciona con la cualidad del software llamada “usabilidad”. ¿Cómo la medimos?

1 Tiempo de aprendizaje

Cuánto tardan usuarios nuevos en completar tareas básicas.

2 Tiempo de ejecución

Cuánto lleva completar una tarea.

3 Tasa de errores

Cuántas veces se equivoca el usuario.

4 Satisfacción

Encuestas al usuario.

5 Retención

Usuarios activos en intervalos de tiempo.

Herramientas

Google Analytics, Quantum Metric y otras; SUS para encuestas.

Pruebas de seguridad

1

Identificar vulnerabilidades

en el software, la red o la infraestructura.

2

Vulnerar los controles

autenticación, autorización, cifrado, auditoría.

3

Verificar el cumplimiento

de políticas, estándares y normativas.

NORMAS Y ESTÁNDARES DE REFERENCIA

OWASP

Top 10 de riesgos web

PCI DSS

Datos de tarjetas

Ley 25.326

Datos personales (Argentina)

Pentesting: simular un ataque real

Dentro de las pruebas de seguridad, el pentesting simula un ataque real a la aplicación o a la infraestructura.

1. **Reconocimiento**
2. **Escaneo**
3. **Explotación**
4. **Post-explotación**
5. **Informe**

Caja negra

El auditor no recibe información previa.

Caja gris

Recibe información parcial, como un usuario común.

Caja blanca

Recibe acceso al código y a la arquitectura.

Requiere autorización escrita y un alcance acordado antes de empezar.

Otros tipos de pruebas de sistema

1 Configuración

Distintas configuraciones de hardware, software, navegadores y sistemas operativos.

2 Compatibilidad y conversión

Convivencia con versiones anteriores y migración de datos.

3 Facilidad de instalación

Instalar, actualizar y desinstalar sin errores.

4 Recuperación (recovery)

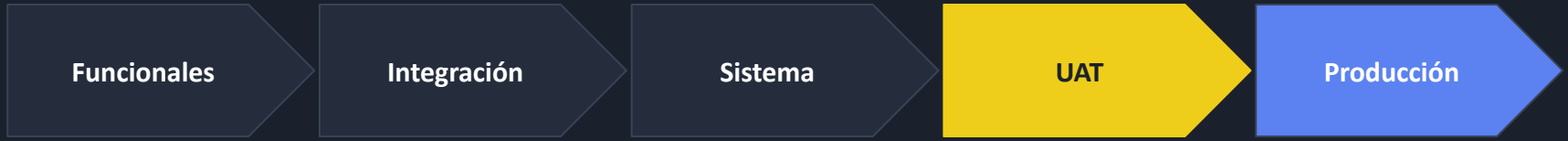
Volver a operar tras una caída, con respaldos y tiempos acordados.

5 Documentación

Manuales y ayuda coinciden con lo que hace el sistema.

NIVEL · ACEPTACIÓN

Pruebas de aceptación del usuario (UAT)



Qué es

Prueba realizada por el usuario final o el cliente para verificar y aceptar el sistema antes de pasarlo a producción.

Se hace en la fase final de pruebas, después de las funcionales, de integración y de sistema.

Dónde y con qué datos

En un entorno de prueba independiente, con una configuración de datos similar a la de producción.

Variantes: aceptación contractual, alfa (dentro de la organización) y beta (con usuarios reales).

Criterios de aceptación: ejemplo

Los criterios de aceptación se escriben antes de construir, con el cliente, en lenguaje del negocio.

```
Escenario: alta de un huésped nuevo
  Dado un huésped que no está registrado
  Cuando el conserje completa el alta con datos válidos
  Entonces el sistema registra al huésped
  Y muestra el mensaje de alta exitosa
```

Dado: el contexto inicial

Cuando: la acción del usuario

Entonces: el resultado observable

NIVEL · ACEPTACIÓN

Pruebas de instalación

Buscan errores de instalación, actualización y desinstalación del sistema. Es la última prueba y no se asocia a una fase específica del proceso.

1 Rutas de instalación

Por defecto y personalizadas.

2 Permisos de usuario

Por ejemplo, instalador ejecutado como administrador.

3 Dependencias

Librerías, drivers, frameworks.

4 Mensajes de error

Claros si falta algún requisito.

5 Reinicio del sistema

Cuando sea necesario.

6 Rollback

Si la instalación falla, volver al estado anterior.

¿Cuándo terminar las pruebas?

CRITERIOS DE POCO SENTIDO

- ✕ Terminar cuando se acaba el tiempo.
- ✕ Terminar cuando se ejecutan todos los casos sin detectar errores.
- ✕ Terminar cuando los casos de valores límite y conjetura de errores no encuentran nada.

CRITERIOS EN TÉRMINOS POSITIVOS

Se hallaron X errores

Se estima cuántos errores hay y se termina al encontrar cierta proporción.

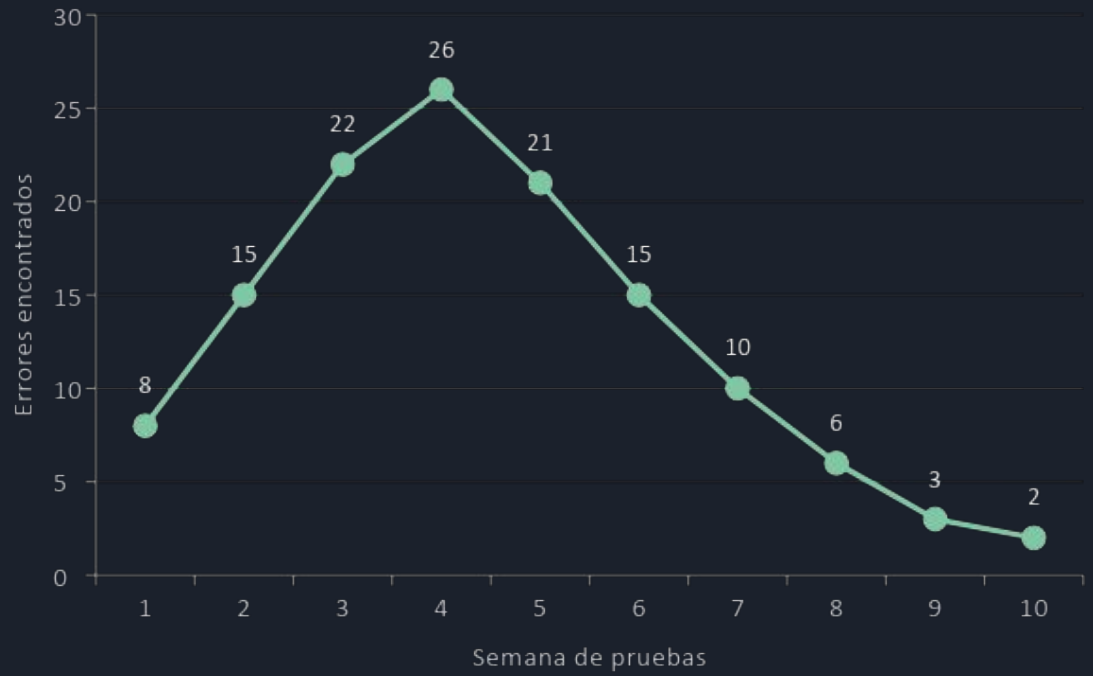
La curva se aplana

Los errores por unidad de tiempo bajan y se estabilizan.

Objetivos de calidad

Cobertura alcanzada y cero defectos críticos abiertos.

La curva de errores encontrados



Cómo leerla

- Sube: se están preparando y ejecutando las pruebas
- Baja y se aplana: cada vez cuesta más encontrar errores
- Ahí aparece el candidato a terminar

PLANIFICACIÓN

Planeamiento y control de la prueba

1	**Objetivos**	Qué se quiere lograr y qué se prueba.
2	**Criterios de terminación**	Cuándo se considera terminada cada fase.
3	**Cronograma**	Fechas, hitos y dependencias.
4	**Responsabilidades**	Quién diseña, ejecuta y revisa.
5	**Asignación de recursos**	Personas, entornos, datos y herramientas.

PRÁCTICA

¿Qué tipo de prueba es?

1. Tras corregir el cálculo del descuento, se vuelven a ejecutar las pruebas de todo el carrito.
2. En cada despliegue se verifica que la app abra, el login funcione y se pueda buscar un producto.
3. Con JMeter se simulan 5.000 usuarios iniciando sesión en un minuto.
4. Se carga una base con 20 millones de huéspedes para ver si el sistema sigue funcionando.
5. Un grupo de recepcionistas valida en un entorno de prueba que el sistema sirve para su trabajo diario, antes de salir a producción.
6. Se prueba el módulo de reservas junto con el de pagos, que ya se habían probado por separado.

Para cada situación: nivel o tipo de prueba, y por qué.

SOLUCIÓN

Solución: ¿qué tipo de prueba es?

#	Tipo	Justificación
1	**Regresión**	Se verifica que un cambio no rompió lo que funcionaba.
2	**Humo**	Verificación mínima de lo básico en cada versión.
3	**Esfuerzo**	Pico de carga en poco tiempo (sería performance si hubiera un tiempo objetivo).
4	**Volumen**	Gran cantidad de datos almacenados.
5	**Aceptación (UAT)**	Usuarios finales validan en un entorno de prueba antes de producción.
6	**Integración**	Módulos ya probados por separado se prueban juntos.

CIERRE

Para llevarse

1

Niveles, tipos y técnicas

Tres preguntas distintas: dónde, qué y cómo.

2

De la unidad a la aceptación

Cada nivel se verifica contra su etapa.

3

Integración incremental

La falla apunta al último módulo agregado.

4

Humo y regresión

Rápidas y automatizadas, en cada cambio.

5

Sistema: no funcionales

Volumen, esfuerzo, performance, seguridad y usabilidad.

6

Definir cuándo terminar

Con criterios positivos y acordados de antemano.